Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —

C2 General

Agenda da Formação

01. Spring Boot & Rest Básico

02. API Design, DTOs & Validação

03. Persistência com JPA & Hibernate

04. Integração Externa (TMDB)

05. Qualidade, Erros & Testes

06. Spring Security, JWT & OAuth2

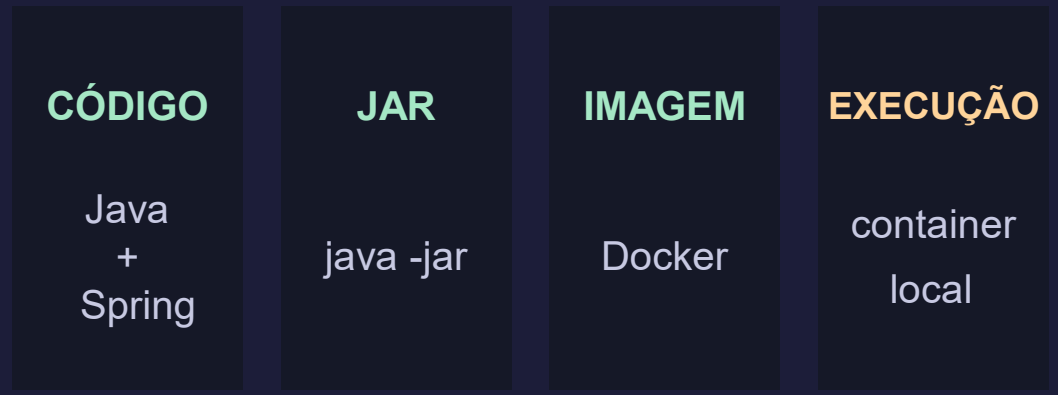
07. Empacotamento & Container

08. Kubernetes Local (Minikube)

Rumos

MyMovies: execução reproduzível

API Spring Boot com PostgreSQL e Docker Compose



Aula 07: da API construída à execução

- Retomar a API protegida da Aula 06
- Configuração externa e segredos
- Docker aplicado à MyMovies
- PostgreSQL e persistência
- Execução local e com Compose
- Validação HTTP e laboratórios

Objetivo: executar a API fora da IDE e em container, com PostgreSQL persistente.

Requisitos antes da instalação



Sistema e processador

SO suportado e pacote para x64 ou ARM64.

Virtualização e recursos

Virtualização quando o backend exige.

Memória RAM e disco para IDE e runtime.

Rede e permissões

Acesso aos downloads e registries.

Proxy e permissões aprovados pela organização.

Os mínimos variam por SO e runtime. A instalação respeita a aprovação e a licença aplicável.

A API está construída. Como a executar noutro ambiente?

PONTO DE PARTIDA

API com Controller, Service e Repository.

Persistência, integração TMDB e segurança já trabalhadas.

DESAFIO DESTA AULA

Executar a mesma API com configuração externa.

Usar PostgreSQL, Compose e validar HTTP e persistência.

A API mantém o contrato. O ambiente de execução muda.

CONCEPT

Configuração externa: o mesmo JAR em vários ambientes

application.properties

Valores por defeito:

porta, URL e opções da aplicação.

Variáveis de ambiente

Valores no arranque.

Exemplo: TMDB_TOKEN.

Argumentos no arranque

Exemplo:

```
--server.port=8081
```

Mudar a configuração dispensa alterar as classes Java.

Segredos: valores externos ao código e à imagem

```
server.port=${SERVER_PORT:8080}
tmdb.token=${TMDB_TOKEN:}
```

DOIS TOKENS, DOIS USOS

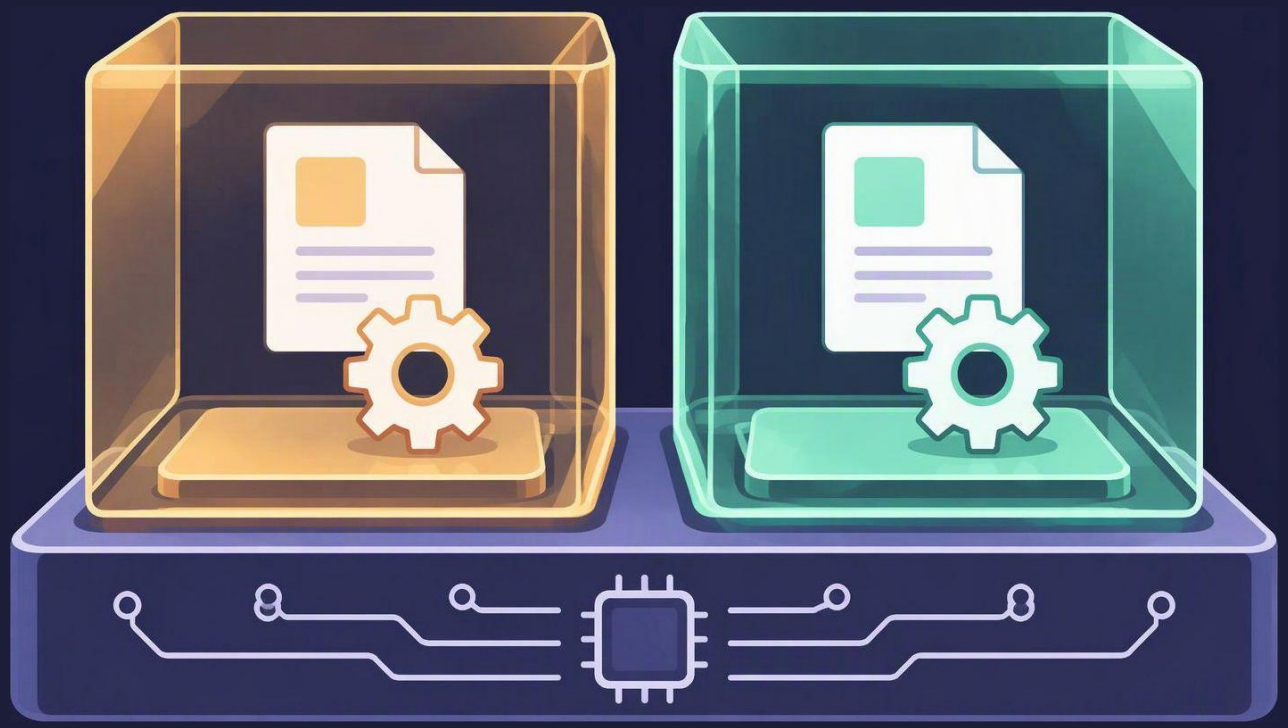
TMDB_TOKEN: a MyMovies usa-o para consultar a TMDB.

JWT do aluno: segue no pedido HTTP para a nossa API.

Nenhum valor real entra no Git ou no Dockerfile.

O lab usa .env.local, fora do Git e do build.

Containers e máquinas virtuais



Container

Processo isolado.

Partilha o núcleo do sistema operativo com outros containers.

Máquina virtual

Executa um sistema operativo com o seu próprio núcleo.

As duas caixas representam containers sobre um núcleo partilhado.

No Docker Desktop, podem executar dentro de uma VM Linux.

PROBLEM

Container: executar a MyMovies num ambiente isolado

APLICAÇÃO

O nosso JAR Spring Boot.

O código da API continua igual.

IMAGEM

Pacote com o JAR, Java 21 e ficheiros necessários ao arranque.

CONTAINER

Uma instância desse pacote.

O processo Java executa num ambiente isolado.

CONCEPT

Imagem e container: pacote e instância

IMAGEM

my-movies:aula7

O nome e a tag identificam o pacote.

Permanece após parar o container.



CONTAINER

Serviço api no Compose.

Nome físico gerado pelo projeto.

Processo Java com configuração própria.

A mesma imagem pode criar vários containers.

Rumos

O caminho de um comando Docker



Cliente / CLI

Terminal: envia o comando do aluno.

Engine

Engrenagem: gere imagens e containers.

Imagem

Pacotes no topo: imagens guardadas.

Container

Caixas em baixo: instâncias em execução.

Docker: a ferramenta que gere containers

Docker CLI

É o comando docker no terminal.

Pedimos: construir uma imagem ou iniciar um container.

Docker Engine

É o serviço que recebe os comandos e gere imagens e containers.

Tem de estar disponível no ambiente aprovado.

Registry: local onde se obtêm ou publicam imagens.

No lab, obtemos as bases Java/PostgreSQL e construímos a imagem da API.

Java no host e Java na imagem

JAR no computador

```
java -jar app.jar
```

O computador fornece o runtime Java.

O processo usa a configuração local.

JAR no container

```
docker run imagem
```

A imagem fornece o runtime Java.

O Engine inicia o processo isolado.

Para construir a MyMovies neste lab, o aluno mantém JDK 21 e Maven Wrapper no host.

Ferramentas do curso: IntelliJ, Git, Maven Wrapper e cliente HTTP.

Mapa da aula: da teoria ao laboratório

Cinco partes: cada conceito vem antes do laboratório que o usa.

Build e imagem

1. Lab 0: build e JAR standalone
2. PostgreSQL e Compose (Lab 1)
3. Dockerfile: a imagem da API (Lab 2)

Ligar e persistir

4. API + PostgreSQL com Compose (Lab 3)
5. Persistência (Lab 4)

Método: conceito, exemplo, laboratório e prova por HTTP.

Parte 1: Lab 0: build e execução do JAR

Antes de conteinerizar, provar que a API corre fora da IDE.

Conceitos

Terminal Git Bash (Windows) ou nativo

Portas 5432, 8081 e 8080 livres

finalName fixo no pom.xml

Laboratório 0

Limpar target/ e gerar o JAR

java -jar e health em localhost:8080

O JAR autónomo é o ponto de partida do container.

LAB 0: SANITY CHECK

Build no Host & Execução Standalone (java -jar)

Validação de paridade da API MyMovies antes da conteinerização Docker

Git Bash / Terminal embutido do IntelliJ (Alt + F12)

```
# 1. Limpar target com segurança (evita erros de lock do Windows/IDE)
rm -rf target/

# 2. Compilar e gerar JAR com nome fixo (finalName=my-movies)
./mvnw package -DskipTests

# 3. Confirmar geração do artefato
ls -lh target/my-movies.jar

# 4. Teste de Execução Standalone (Fora da IDE)
java -jar target/my-movies.jar

# 5. Teste HTTP em outro terminal / browser:
curl -i http://localhost:8080/actuator/health
# Retorno esperado: HTTP/1.1 200 OK {"status":"UP"}

# 6. Encerrar aplicação no terminal: Ctrl + C
```

Pré-Configuração IDE

- Windows: Ajustar terminal no IntelliJ:
  Settings > Tools > Terminal > Shell path
  Selecionar C:\Program Files\Git\bin\bash.exe
- macOS / Linux: Terminal nativo (Zsh/Bash)
- Vantagem: Sintaxe única para toda a turma!

Definition of Done (DoD)

[x] <finalName>my-movies</finalName>
adicionado em <build> do pom.xml

[x] Target limpo com 'rm -rf target/'

[x] JAR compilado: target/my-movies.jar

[x] Teste 'java -jar' respondeu 200 OK
no endpoint /actuator/health

[x] Aplicação finalizada (Ctrl+C) e
portas 5432 e 8081 desocupadas

Dica do Instrutor: Ao executar rm -rf target/ no Git Bash, evita-se qualquer trava de arquivo do Windows antes de empacotar.

Parte 2: PostgreSQL e Compose (Lab 1)

Primeiro a base de dados: o Compose descreve e inicia o serviço db.

Conceitos

Compose e o ficheiro compose.yaml

Serviços, rede e volumes

Valores por defeito e variáveis

Laboratório 1

Escrever o serviço db

Confirmar que fica saudável

Começamos pela base de dados; a API entra na Parte 4.

H2 e PostgreSQL: a mudança na MyMovies

H2 em memória

Base embutida no processo Java.

O modo em memória perde os dados quando a aplicação termina.

PostgreSQL no container

Servidor de base de dados separado.

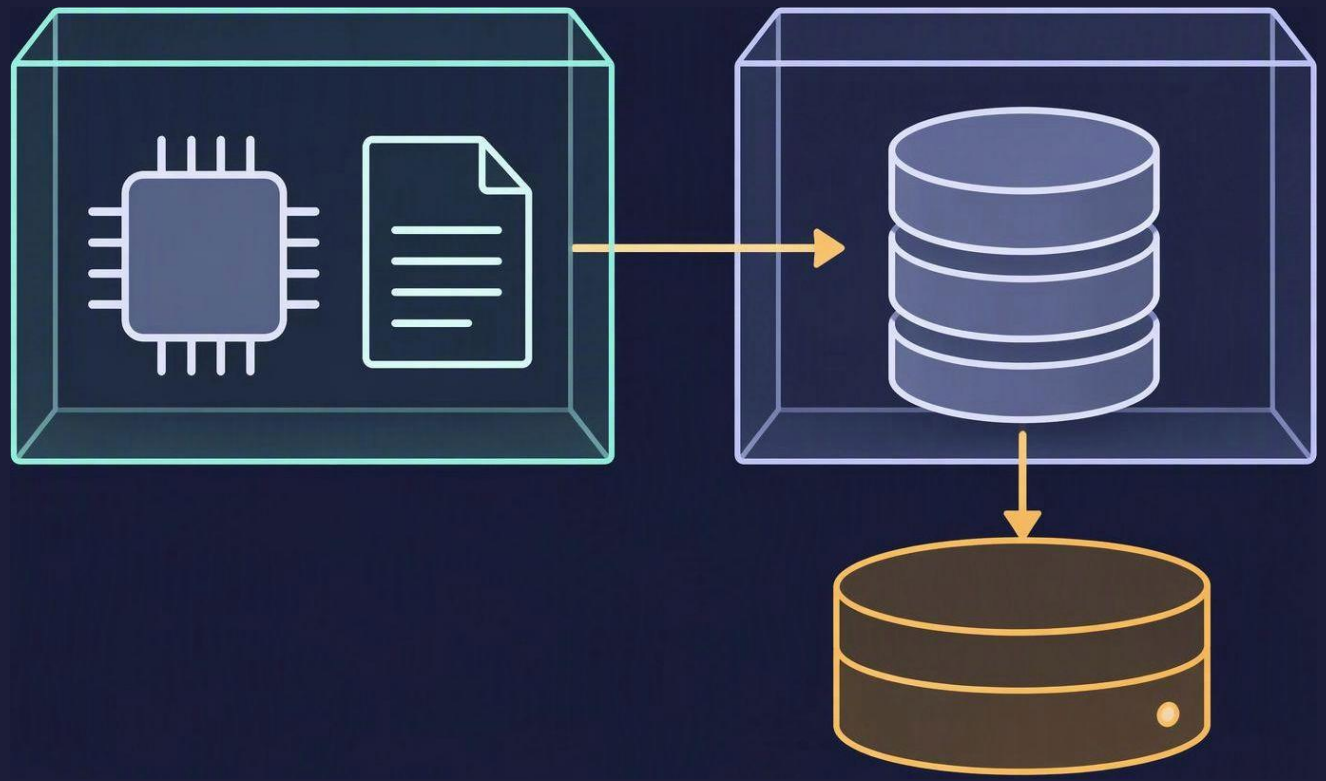
A API liga-se por JDBC.

Um volume conserva os dados.

O contrato HTTP mantém-se. Mudam o driver, a configuração e a base de dados.

Este lab começa com uma base nova. Não copia automaticamente os dados do H2.

API e PostgreSQL em serviços separados



A API envia pedidos SQL a db.

Só db lê e escreve no volume pgdata.

api

JAR Spring Boot e runtime Java.

Imagem construída pelo Dockerfile.

db

Servidor PostgreSQL.

Imagem oficial postgres:16.

pgdata

Volume que guarda os ficheiros da base.

Só o serviço db o monta.

Docker Compose: a aplicação com vários serviços

Um ficheiro descreve como executar a API e a base de dados em conjunto.

Na MyMovies

api executa o Java Spring Boot.

db executa o PostgreSQL.

A rede permite a comunicação.

pgdata conserva os dados da base.

```
docker compose up -d
```

Compose lê compose.yaml e pede ao Docker Engine para criar e iniciar os serviços definidos.

Compose v2 usa o comando docker compose. O runtime aprovado deve disponibilizá-lo.

Compose: leitura dos serviços

Campo	Significado	Leitura do exemplo
services	Agrupa os componentes da aplicação.	`api: ...` `db: ...`
image / build	image escolhe o pacote. build indica como construí-lo.	`db: image: postgres:16` `api: build: .`
environment **env_file**	Configuração enviada ao processo do container.	`SPRING_PROFILES_ACTIVE: postgres` `env_file: .env.local`
ports	Publica a porta do container no computador.	`127.0.0.1:8081:8080` `host local 8081, API 8080`

YAML usa indentação com espaços. api e db ficam no mesmo nível dentro de services.

Compose: rede, persistência e arranque

Elemento	Significado	No nosso lab
name / rede	name identifica o projeto. A rede liga os serviços.	`name: mymovies-aula7` `A API encontra a base por db.`
volumes	O serviço monta o volume. O nível principal declara-o.	`db monta pgdata.` `volumes: declara pgdata.`
healthcheck **depends_on**	O teste mede prontidão. A dependência ordena o arranque.	`db usa pg_isready.` `api espera service_healthy.`

up cria/inicia. ps mostra estado. logs ajuda no diagnóstico. down remove serviços e rede.

PostgreSQL: os dados ficam no volume

CONTAINER db

Executa o servidor PostgreSQL.

Pode parar ou ser recriado.

A imagem não contém o catálogo do aluno.

VOLUME pgdata

Guarda os ficheiros da base.

down conserva-o. down -v apaga-o.

Credenciais iniciais só atuam quando o volume está vazio.

Volume não substitui um backup.

compose.yaml: serviço PostgreSQL

```yaml
name: mymovies-aula7
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_DB: mymovies
      POSTGRES_USER: ${POSTGRES_USER:-mymovies_lab}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-mymovies_pass}
    ports: ["127.0.0.1:5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
```

db identifica o serviço. A imagem fornece PostgreSQL. pgdata conserva os dados.

compose.yaml: readiness e volume

Continuação dentro do serviço db

```
    healthcheck:
      test:
        - CMD-SHELL
        - >-
          pg_isready -h 127.0.0.1
          -U ${POSTGRES_USER} -d ${POSTGRES_DB}
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s

volumes:
  pgdata:
```

Nesta fase, compose.yaml contém só db e pgdata.

api será acrescentada depois.

volumes fica no nível principal.

pg_isready testa o servidor por TCP.

Lab 1: PostgreSQL solitário em container

```
docker compose version
docker compose up -d db
docker compose ps
docker compose logs db
```

Confirmar

Compose disponível.

Serviço db (healthy).

Porta 5432 em 127.0.0.1.

Logs sem erros.

Não é preciso instalar o servidor PostgreSQL no host. O container fornece-o.

Pré-requisito: compose.yaml com db, pgdata e healthcheck.

LAB 1: PostgreSQL Solitário em Container

Criar o serviço de banco de dados isolado no Docker Compose

ARQUIVO: compose.yaml (raiz do projeto)

```
services:
  db:
    image: postgres:16
    container_name: mymovies-db
    environment:
      POSTGRES_DB: mymovies
      POSTGRES_USER: mymovies_lab
      POSTGRES_PASSWORD: mymovies_pass
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL",
             "pg_isready -h 127.0.0.1 -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s

volumes:
  pgdata:
```

TERMINAL (Git Bash / macOS)

```
# 1. Subir apenas o serviço db:
docker compose up -d db

# 2. Inspecionar status e healthcheck:
docker compose ps

# 3. Verificar logs do container PostgreSQL:
docker compose logs -f db
```

DEFINITION OF DONE (DoD)

- [X] Container 'db' rodando com status (healthy)
- [X] Porta 5432 livre e vinculada a 127.0.0.1
- [X] Volume nomeado 'pgdata' criado pelo Docker

Parte 3: Docker: da imagem ao Dockerfile (Lab 2)

O Lab 0 deu-nos o JAR. Agora aprendemos a construir a imagem da API.

Conceitos

Dockerfile: a receita da imagem

FROM, WORKDIR, COPY, EXPOSE, USER

ENTRYPOINT e CMD

Contexto de build e .dockerignore

Laboratório 2

Construir o JAR com Maven

Construir a imagem com docker build

Ainda sem Compose: só a imagem da API.

Dockerfile: as instruções para construir a imagem

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/my-movies.jar app.jar
EXPOSE 8080
USER 10001
ENTRYPOINT ["java","-jar","/app/app.jar"]
```

FROM: runtime Java 21.

WORKDIR: pasta de trabalho.

COPY: copia o JAR construído.

EXPOSE: documenta a porta.

USER: executa sem ser root.

ENTRYPOINT: arranca a API.

PostgreSQL vem de outra imagem.

Dockerfile: base, pasta e ficheiros

Instrução	O que faz	Exemplo e efeito
FROM	Escolhe a imagem base de uma etapa de construção.	`FROM eclipse-temurin:21-jre` `Disponibiliza o runtime Java 21.`
WORKDIR	Define a pasta para as instruções seguintes e o arranque.	`WORKDIR /app` `Cria a pasta se ela não existir.`
COPY	Copia ficheiros do contexto para o sistema de ficheiros da imagem.	`COPY target/my-movies.jar app.jar` `Destino: /app/app.jar`

COPY exige o JAR já construído. Neste Dockerfile, Maven executa antes do docker build.

Dockerfile: porta, utilizador e saúde

Instrução	O que configura	Exemplo e cuidado
EXPOSE	Documenta a porta usada pela aplicação.	`EXPOSE 8080` `ports` no Compose publica-a no host.
USER	Define o utilizador/UID das próximas ações.	`USER 10001` O processo precisa de permissões nas pastas em que escreve.
HEALTHCHECK	Comando periódico para verificar o container.	Opcional na imagem da API. O lab define o teste de db em `compose.yaml`.

O healthcheck de db verifica PostgreSQL. A resposta HTTP verifica a aplicação Spring Boot.

ENTRYPOINT e CMD: o que arranca

ENTRYPOINT

Define o executável principal

e os seus argumentos fixos.

CMD

Fornece comando ou argumentos

por defeito, substituíveis no arranque.

Exemplo didático. O Dockerfile do lab mantém apenas ENTRYPOINT.

```
ENTRYPOINT ["java","-jar","/app/app.jar"]
CMD ["--server.port=8080"]
```

```
Por defeito: java -jar /app/app.jar --server.port=8080
```

Com ENTRYPOINT, os argumentos no arranque substituem CMD e seguem depois do executável.

Contexto do build e .dockerignore

```
docker build -t my-movies:aula7 .
```

O ponto seleciona a pasta do projeto como contexto.

COPY só vê os ficheiros disponíveis nesse contexto.

.dockerignore

```
.env
.env*
.git/
.idea/
```

Neste lab

Manter o JAR necessário em target/.

Excluir os ficheiros com segredos.

Evitar COPY . . sem avaliar o conteúdo.

Lab 2: Dockerfile e imagem da API

```
./mvnw package
docker build -t my-movies:aula7 .
docker images | grep my-movies
```

O que confirmar

JAR em target/my-movies.jar.

Imagem my-movies:aula7 criada.

Tamanho listado em docker images.

A API é a mesma. O container fornece o Java 21 e corre com USER 10001.

Ainda sem PostgreSQL: a API só se liga à base no Lab 3.

LAB 2: Dockerfile & Imagem da API

Empacotamento da aplicação Spring Boot em imagem portátil e segura

ARQUIVO: Dockerfile (raiz do projeto)

```
FROM eclipse-temurin:21-jre
WORKDIR /app

# Copia o JAR padronizado gerado no Lab 0
COPY target/my-movies.jar app.jar

# Exposição de porta documentacional
EXPOSE 8080

# Boa Prática Sec: Execução com usuário não-root
USER 10001

# Entrypoint de inicialização da JVM
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

TERMINAL (Git Bash / macOS)

```
# 1. Gerar a imagem Docker da API:
docker build -t my-movies:aula7 .

# 2. Verificar a imagem criada localmente:
docker images | grep my-movies

# 3. Teste rápido de inspeção da imagem:
docker inspect my-movies:aula7
```

DEFINITION OF DONE (DoD)

- [X] Dockerfile criado na raiz com JRE 21 enxuta
- [X] Imagem 'my-movies:aula7' compilada com sucesso
- [X] Segurança ativada: usuário 10001 (não-root)

Parte 4: API e PostgreSQL com Compose (Lab 3)

A base e a imagem já existem.

Agora a API liga-se ao PostgreSQL na mesma rede.

Conceitos

Driver JDBC e perfil postgres

Tabelas, dados e testes

Segredos em .env.local

Serviço api no Compose, portas e health

Laboratório 3

Lab 3: api e db com Compose

Validação HTTP em localhost:8081

Mesma API, mesmo contrato: muda onde executa.

pom.xml: driver PostgreSQL

Dentro de <dependencies>

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

H2 sai do runtime da aplicação

JDBC

O driver permite à aplicação Java comunicar com PostgreSQL.

Spring Data JPA permanece.

Spring Boot: perfil postgres

`src/main/resources/application-postgres.properties`

Perfil: conjunto de propriedades ativado para um ambiente.

```
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${POSTGRES_USER}
spring.datasource.password=${POSTGRES_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver
spring.jpa.hibernate.ddl-auto=update
spring.sql.init.mode=never
spring.h2.console.enabled=false
```

Ativação: SPRING_PROFILES_ACTIVE=postgres

Remover o dialect H2 legado. update serve o lab. Produção usa migrações e validate.

Tabelas, dados iniciais e testes

Tabelas no PostgreSQL

No lab novo, Hibernate update usa as entidades para preparar o esquema.

Dados iniciais

Scripts H2 e SQL nativo podem precisar de adaptação. Os dados antigos não aparecem na nova base.

Testes

Rever o perfil de teste. Testes de integração devem validar PostgreSQL numa base separada.

Um volume preserva ficheiros. Migrações e backups continuam a ser necessários.

Segredos: .env.local

Sem .env: valores por defeito

```
POSTGRES_USER=mymovies_lab
POSTGRES_PASSWORD=mymovies_pass
```

O YAML usa ${VAR:-defeito}.

Um .env opcional substitui-os.

.env.local

```
TMDB_TOKEN=seu_bearer_token_tmdb_aqui
# criar na raiz, fora do Git
```

env_file envia a configuração

da API ao container api.

.env.local fica fora do Git e do build. Com o Bruno, usar a porta 8081.

Dockerfile e compose.yaml: responsabilidades

Ficheiro	O que define	Exemplo na MyMovies
Dockerfile	Como construir uma imagem.	`Java 21 + JAR da API`
compose.yaml	Como executar os serviços e ligar os recursos.	`api, db, portas e pgdata`
.env	Valores para preencher ${...} no ficheiro Compose.	`POSTGRES_USER` `POSTGRES_PASSWORD`
.env.local	Variáveis enviadas a api pelo campo env_file.	`TMDB_TOKEN e configuração exigida pelo README`

A imagem PostgreSQL já existe. Neste lab, só a imagem da API precisa do nosso Dockerfile.

compose.yaml: serviço da API

```yaml
api:
  build: .
  image: my-movies:aula7
  env_file: .env.local
  environment:
    SPRING_PROFILES_ACTIVE: postgres
    SPRING_DATASOURCE_URL: jdbc:postgresql://db:5432/mymovies
    POSTGRES_USER: ${POSTGRES_USER:-mymovies_lab}
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-mymovies_pass}
  ports: ["127.0.0.1:8081:8080"]
  depends_on:
    db:
      condition: service_healthy
```

Acrescentar api em services, ao lado de db. Manter um único bloco volumes no final.

Portas e endereços: host e containers

```
api: 127.0.0.1:8081:8080
```

Cliente HTTP no computador:
http://localhost:8081
API dentro do container: 8080

IDE/JAR liga ao PostgreSQL:
localhost:5432
API em container liga a db:5432

```
localhost depende de onde o processo executa
```

Dentro de api, localhost aponta para api. O nome **db** identifica PostgreSQL na rede Compose.
A porta 5432 publicada permite acesso local pela IDE, sem expor a base à rede externa.

Health e logs: sinais da execução da API

JAR

`localhost:8080`
`/actuator/health`

CONTAINER

`localhost:8081`
`/actuator/health`

PROVA FUNCIONAL

GET /api/movies
e POST com JWT válido

Health verifica o estado. Logs ajudam a explicar falhas.

Lab 3: API e db com Compose

```
./mvnw package
docker compose down
docker compose up -d --build

docker compose ps
docker compose logs api
```

package recompõe o JAR.

down desliga os containers ativos.

--build reconstrói a imagem Java.

compose inicia db e api.

api aguarda o healthcheck de db.

API local: http://localhost:8081

Lab 3: validação HTTP e encerramento

Observar

```
docker compose ps
docker compose logs api
```

ps mostra os serviços api e db.

logs api mostra a saída do Java.

Confirmar db saudável.

Provar por HTTP

```
GET http://localhost:8081/actuator/health
GET http://localhost:8081/api/movies
```

Usar o JWT válido no pedido protegido.

Confirmar status e JSON esperado.

```
Retorno esperado: 200 OK, db UP. Segue o Lab 4.
```

LAB 3 Orquestração Multi-Container com Docker Compose (API + DB)

1. pom.xml (Dependência PostgreSQL)

```xml
<dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
</dependency>
```

2. compose.yaml (Visão Integral: db + api + volumes)

```yaml
name: mymovies-aula7

services:
  db:                           # <--- Serviço PostgreSQL (Lab 1)
    image: postgres:16
    environment:
      POSTGRES_DB: mymovies
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports: ["5432:5432"]
    volumes: [pgdata:/var/lib/postgresql/data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d mymovies"]
      interval: 5s
      timeout: 5s
      retries: 5

  api:                          # <--- Serviço API Spring Boot (Lab 3)
    build: .
    ports:
      - "127.0.0.1:8081:8080"
    env_file: .env.local
    environment:
      - SPRING_PROFILES_ACTIVE=postgres
      - SPRING_DATASOURCE_URL=jdbc:postgresql://db:5432/mymovies
      - POSTGRES_USER=${POSTGRES_USER}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD}
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata: {}                     # <--- Volume de Persistência
```

3. Configurações & Ficheiros de Ambiente (.env)

a) src/main/resources/application-postgres.properties:

```
spring.datasource.url=${SPRING_DATASOURCE_URL}
spring.datasource.username=${POSTGRES_USER}
spring.datasource.password=${POSTGRES_PASSWORD}
spring.datasource.driver-class-name=org.postgresql.Driver
```

b) Ficheiro .env (na raiz do projeto - para o Compose):

```
POSTGRES_USER=mymovies_lab
POSTGRES_PASSWORD=mymovies_pass
```

c) Ficheiro .env.local (na raiz do projeto - para a API):

```
TMDB_TOKEN=seu_bearer_token_aqui
POSTGRES_USER=mymovies_lab
POSTGRES_PASSWORD=mymovies_pass
```

4. Sequência Terminal Git Bash (Rebuild & Deploy)

```bash
# Passo 1: Parar containers anteriores
docker compose down

# Passo 2: Limpar target e gerar JAR com driver Postgres
rm -rf target/
./mvnw package -DskipTests

# Passo 3: Reconstruir imagem da API e subir pilha completa
docker compose up -d --build

# Passo 4: Acompanhar os logs do arranque do Spring Boot
docker compose logs -f api
```

Definition of Done (DoD) - Validação Lab 3

- [X] Dependência org.postgresql no pom.xml e driver-class-name setado
- [X] Ficheiros .env e .env.local criados na raiz com as credenciais
- [X] docker compose up -d --build finalizado com código exit 0
- [X] Logs da API exibem: Started MyMoviesApplication in X seconds
- [X] Teste HTTP 200 OK: curl -i http://localhost:8081/actuator/health
- [X] Resposta Health: {"status":"UP","components":{"db":{"status":"UP"...}}}

Diagnóstico rápido do laboratório

Engine indisponível	docker version não contacta o servidor. Confirmar serviço Docker.
JAR ausente no build	Executar ./mvnw package e confirmar o nome real em target/.
Porta ou nome já usados	Consultar docker ps -a. Usar outra porta ou terminar a própria instância.
Falha de ligação ao PostgreSQL	Verificar logs api/db, URL (localhost ou db), credenciais e tabelas.

```
Diagnóstico: docker compose ps -a
```

ROADMAP

Aula 08: o container chega ao Kubernetes local



Aula 08: API e acesso ao PostgreSQL no cluster local.

Rumos

Checkpoint: aula-07-deployable

- ✓ Maven package com testes verdes
- ✓ JAR executável e nome confirmado
- ✓ API por HTTP com PostgreSQL fora da IDE
- ✓ Configuração externa, sem segredos no Git
- ✓ Compose com api e db funcionais
- ✓ Registo preservado após recriar sem -v

`checkpoint/aula-07-deployable`

Resultado: a mesma API executa com configuração externa e dados persistentes.